should add the middleware without a default policy by calling `UseCors()` only.

You can then selectively enable CORS for individual endpoints and specifying the policy to apply. To apply CORS to a minimal API endpoint or route group, call `RequireCors("AllowShoppingApp")`, as shown in the following listing. To apply a policy to a controller or an action method, apply the `[EnableCors("AllowShoppingApp"]` attribute. You can disable cross-origin access for an endpoint by applying the `[DisableCors]` attribute.

Listing 29.4 Applying a CORS policy to minimal API endpoints

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
builder.Services.AddCors(options => { /* Config not shown*/});

var app = builder.Build();
app.UseCors();                                                    ❶

app.MapGet("/api/products", () => new string[] {})
    .RequireCors("AllowShoppingApp");                             ❷

app.MapGet("/api/products",
    [EnableCors("AllowShoppingApp")] () => new { });              ❸

app.MapGroup("/api/categories")
    .RequireCors("AllowAnyOrigin");                               ❹

app.MapDelete("/api/products",
    [DisableCors] () => Results.NoContent());                     ❺

app.Run();
```

❶ Adds the CorsMiddleware without configuring a default policy

❷ Applies the AllowShoppingApp CORS policy to the endpoint

❸ You can apply attributes to the lamba or handler method, as well as to MVC action methods.

❹ You can apply CORS policies to whole route groups.

❺ The DisableCors attribute disables CORS for the endpoint completely.